

CSA-0606- DESIGN AND ANALYSIS OF ALGORITHMS

CO 3 – AT 1

Experiment 30: Divide and Conquer Matrix Multiplication

1. Assessment Question

Problem Statement

Implement **matrix multiplication using the Divide and Conquer technique**. Recursively divide each matrix into smaller submatrices, multiply them, and combine the results. Record the execution time for different matrix sizes and analyze the efficiency of the recursive approach.

Input Requirements

- Two square matrices of the same size.
- Matrix sizes such as **2×2, 4×4, 8×8, 16×16, 32×32**.
- Matrix elements can be randomly generated.

Expected Output

- Product of the two matrices.
- Execution time for each matrix size.
- Comparison of execution times.

Constraints

- Both matrices must have the same dimensions.
- For simple recursive splitting, using powers of 2 is convenient.
- Larger matrices require more computation time.

Assessment Criteria

- Correct recursive division.
- Correct matrix multiplication.

- Execution-time measurement.
- Result analysis.
- Complexity explanation.

2. Objective

To implement **Divide and Conquer Matrix Multiplication** using recursive splitting of matrices and evaluate its performance for different matrix sizes.

3. Required Resources

Hardware

- Computer/Laptop
- Minimum 4 GB RAM

Software

- Python 3.x
- Python IDLE / VS Code

Programming Language

- Python

Dataset/Input Files

- No external dataset is required.
- Matrices are generated automatically by the program.

4. Algorithm / Procedure

1. Generate two square matrices.
2. Divide each matrix into four smaller matrices.
3. Recursively multiply the corresponding submatrices.
4. Add the resulting submatrices.

5. Continue dividing until the matrix size becomes 1×1 .
6. Combine the smaller results to obtain the final matrix.
7. Measure the execution time.
8. Repeat for different matrix sizes.
9. Compare the execution times.

Simple Flow

Original Matrices



Divide into 4 parts



Recursively multiply



Combine the results



Final Matrix



Measure Execution Time

5. Implementation

Below is a **simple Python program that can be directly run in IDLE.**

```
import random
```

```
import time
```

```
# Add two matrices
```

```
def add_matrix(A, B):
```

```
n = len(A)
return [[A[i][j] + B[i][j] for j in range(n)] for i in range(n)]
```

Divide and Conquer Matrix Multiplication

```
def multiply(A, B):
```

```
    n = len(A)
```

```
    # Base case
```

```
    if n == 1:
```

```
        return [[A[0][0] * B[0][0]]]
```

```
    mid = n // 2
```

```
    # Divide A into 4 parts
```

```
    A11 = [row[:mid] for row in A[:mid]]
```

```
    A12 = [row[mid:] for row in A[:mid]]
```

```
    A21 = [row[:mid] for row in A[mid:]]
```

```
    A22 = [row[mid:] for row in A[mid:]]
```

```
    # Divide B into 4 parts
```

```
    B11 = [row[:mid] for row in B[:mid]]
```

```
    B12 = [row[mid:] for row in B[:mid]]
```

```
    B21 = [row[:mid] for row in B[mid:]]
```

```
    B22 = [row[mid:] for row in B[mid:]]
```

```
# Recursive multiplication
```

```
C11 = add_matrix(multiply(A11, B11),  
                 multiply(A12, B21))
```

```
C12 = add_matrix(multiply(A11, B12),  
                 multiply(A12, B22))
```

```
C21 = add_matrix(multiply(A21, B11),  
                 multiply(A22, B21))
```

```
C22 = add_matrix(multiply(A21, B12),  
                 multiply(A22, B22))
```

```
# Combine four parts
```

```
C = []
```

```
for i in range(mid):
```

```
    C.append(C11[i] + C12[i])
```

```
for i in range(mid):
```

```
    C.append(C21[i] + C22[i])
```

```
return C
```

```
# Generate a random matrix
```

```
def create_matrix(n):
```

```
    return [[random.randint(1, 9) for j in range(n)]  
            for i in range(n)]
```

```
# Test different matrix sizes
```

```
sizes = [2, 4, 8, 16, 32]
```

```
print("DIVIDE AND CONQUER MATRIX MULTIPLICATION")
```

```
print("-----")
```

```
for n in sizes:
```

```
    A = create_matrix(n)
```

```
    B = create_matrix(n)
```

```
    start = time.perf_counter()
```

```
    C = multiply(A, B)
```

```
    end = time.perf_counter()
```

```
    execution_time = end - start
```

```
    print("Matrix Size:", n, "x", n)
```

```
    print("Execution Time:", execution_time, "seconds")
```

```
# Display result only for 2x2 matrix
```

```

if n == 2:

    print("Matrix A:", A)

    print("Matrix B:", B)

    print("Result:", C)

print()

```

OUTPUT:

```

Programiz Python Online Compiler
main.py
1 import random
2 import time
3
4
5 # Add two matrices
6 def add_matrix(A, B):
7     n = len(A)
8     return [[A[i][j] + B[i][j] for j in range(n)] for i in range(n)]
9
10
11 # Divide and Conquer Matrix Multiplication
12 def multiply(A, B):
13
14     n = len(A)
15
16     # Base case
17     if n == 1:
18         return [[A[0][0] * B[0][0]]]
19
Output
DIVIDE AND CONQUER MATRIX MULTIPLICATION
-----
Matrix Size: 2 x 2
Execution Time: 1.874999998108251e-05 seconds
Matrix A: [[7, 3], [8, 3]]
Matrix B: [[8, 7], [3, 6]]
Result: [[65, 67], [73, 74]]

Matrix Size: 4 x 4
Execution Time: 9.678999958850909e-05 seconds

Matrix Size: 8 x 8
Execution Time: 0.0006840200003352948 seconds

Matrix Size: 16 x 16
Execution Time: 0.006524966999677417 seconds

Matrix Size: 32 x 32
Execution Time: 0.0363048279996292 seconds

```

Important Functions

Function	Purpose
create_matrix()	Creates a random matrix
add_matrix()	Adds two matrices
multiply()	Performs recursive multiplication
time.perf_counter()	Measures execution time

6. Input and Output

The program automatically generates matrices, so no manual input is required.

Sample Input

For example, for size **2×2**:

Matrix A = [[2, 3],
[4, 5]]

Matrix B = [[1, 2],
[3, 4]]

Sample Output

DIVIDE AND CONQUER MATRIX MULTIPLICATION

Matrix Size: 2 x 2

Execution Time: 0.00004 seconds

Matrix A: [[2, 3], [4, 5]]

Matrix B: [[1, 2], [3, 4]]

Result: [[11, 16], [19, 28]]

Matrix Size: 4 x 4

Execution Time: 0.00015 seconds

Matrix Size: 8 x 8

Execution Time: 0.00120 seconds

Matrix Size: 16 x 16

Execution Time: 0.00950 seconds

Matrix Size: 32 x 32

Execution Time: 0.07100 seconds

Note: Your exact execution times will be different depending on your computer.

7. Results

You can record your actual output in the following table:

Matrix Size	Execution Time (seconds)
2×2	0.00004
4×4	0.00015
8×8	0.00120
16×16	0.00950
32×32	0.07100

Observation

As the matrix size increases, the execution time also increases.

A simple graph can be made with:

- **X-axis:** Matrix size
- **Y-axis:** Execution time

The graph will generally show an increasing curve.

8. Analysis of Results

Expected Output

The expected matrix multiplication result was obtained correctly. The program recursively divides the matrices and combines the smaller results.

Performance

The execution time increases as the matrix size increases because more recursive multiplication operations are required.

Time Complexity

For this basic Divide and Conquer multiplication:

Therefore:

So, the basic recursive approach has the **same asymptotic time complexity as conventional matrix multiplication**, although it demonstrates the divide-and-conquer strategy clearly.

Space Complexity

The recursive calls and temporary submatrices require additional memory.

Approximately:

space is required for storing matrices, with additional recursive overhead.

Advantages

- Breaks a large problem into smaller problems.
- Easy to understand recursively.
- Demonstrates the Divide and Conquer technique.
- Smaller matrices can be processed independently.
- Provides a foundation for more advanced algorithms such as **Strassen's algorithm**.

Limitations

- Basic recursive multiplication does **not reduce the asymptotic complexity below $O(n^3)$** .
- Creates many temporary matrices.
- Recursion adds overhead.
- For small matrices, ordinary iterative multiplication may be faster.

Comparison

Method	Time Complexity	Main Idea
Normal Matrix Multiplication	$O(n^3)$	Direct multiplication
Divide and Conquer	$O(n^3)$	Recursively divide matrices

Method	Time Complexity	Main Idea
Strassen's Algorithm	$O(n^{2.81})$ approximately	Uses fewer recursive multiplications

9. Conclusion

The Divide and Conquer Matrix Multiplication algorithm was successfully implemented using recursive splitting. The matrices were divided into smaller submatrices, multiplied recursively, and then combined to produce the final result.

The experiment showed that execution time increases as matrix size increases. The basic recursive method has **$O(n^3)$ time complexity**, similar to traditional matrix multiplication. Therefore, recursive division makes the problem easier to break down and understand, but it does not by itself provide a lower asymptotic complexity. More advanced divide-and-conquer techniques such as **Strassen's algorithm** can provide better theoretical efficiency for large matrices.